

The code, explained

Every script in the project: what it does, how it works, and why

SRON COGNITO post-doc application

Vikrant Tomar · August 2026

How to read this.

There are nine Python files. Three of them you run in order to produce a result; the rest are either helpers the others call, or tools for looking at what came out. Each entry below says what the file is for, walks through what it actually does step by step, and flags the one or two decisions inside it that could reasonably have gone another way.

No prior knowledge of neural networks is assumed.

The nine files, and how they fit together

Three files do the work, in this order:

```
python src/selftest.py           # 1 min, checks the machinery
python src/get_training_data.py  # downloads 418 MB
python src/train.py --data-dir data # 20-45 min, makes plume_cnn.pt
python src/fetch_co_tiles.py --days 18 # 30-40 min, makes co_tiles.npz
python src/predict_co.py        # 2 min, makes co_detections.json
python src/browse_tiles.py --max-score 0.3 # look at what happened
```

File	Kind	What it is for
selftest.py	tool	Invents fake plumes and runs the whole pipeline on them, so you find out in a minute whether your Python setup works.
get_training_data.py	step 1	Downloads the labelled methane pictures that the detector learns from.
dataset.py	helper	Reads those files, strips the units off each picture, and makes the eight rotated and mirrored copies.
model.py	helper	The detector itself — the arrangement of filters and weights.
train.py	step 2	Teaches the detector what a plume looks like, using the methane pictures.
fetch_co_tiles.py	step 3	Downloads carbon monoxide pictures from Google Earth Engine around your sites.
fetch_co_l2.py	optional	The same thing from raw satellite files instead, in the instrument's own geometry. Better science, more setup.
predict_co.py	step 4	Runs the trained detector on the CO pictures and scores every one.
browse_tiles.py	tool	Makes one picture-sheet per site so you can look at the tiles yourself.
facilities.csv	data	The 26 places you look at: 20 steel plants, 3 cities, 3 empty controls.

selftest.py — prove the machinery works

Why it exists. Everything else takes tens of minutes. This takes one, and needs no downloads. If it fails, the problem is your Python installation, not the science, and you have saved yourself an hour of confusion.

How it works

- It **invents** 800 fake pictures. Half are pure noise with a gentle slope across them. The other half are the same, plus a made-up plume: a bright spot at a random position, with a tail trailing off in a random direction, getting wider and fainter as it goes — the way a real plume behaves in wind.
- It splits them into a training set and a test set, trains the real detector on the training half for five short rounds, then scores the half it never saw.
- If it can tell the two apart, every piece of the chain works: the file reading, the unit-stripping, the rotations, the network, the training loop, the scoring.

Expect a perfect score, and do not be impressed by it. The fake negatives are pure noise with no structure at all, so telling them from a plume is trivial. Real negatives contain retrieval artefacts that genuinely look like plumes. A perfect selftest means the plumbing works — nothing more.

The plume generator inside it is reusable

It builds a plume by walking along a line from a source point and adding a blob at each step, each blob a little wider and a little weaker than the last. That is a crude Gaussian plume model. If you ever want to make synthetic training data for CO — one of the three routes to a real CO training set — this function is the seed of it.

get_training_data.py — fetch the labelled examples

What it downloads. Three files from Zenodo, about 418 MB in total, published by Schuit and colleagues under a licence that allows reuse as long as you credit them. Crediting is a condition, not a courtesy.

File	Size	Contents
CNN_pos_trainingdata.nc	88 MB	828 pictures a human marked plume
CNN_neg_trainingdata.nc	239 MB	2,242 pictures marked no plume
SVC_trainingdata.nc	90 MB	843 pictures marked plume / artefact / empty — for the second stage, which this project does not attempt

What was wrong with the first version, and what changed

The original drew its progress bar by returning to the start of the line and overwriting it. That works in a terminal. It does nothing in a notebook, an IDE console, or a log file — so a working download looked frozen. Four changes:

- **Progress that survives anywhere.** In a terminal it still redraws one line. Anywhere else it prints a fresh line every 5%, with transfer speed and an estimate of time remaining.
- **Downloads to a temporary name** and only renames on success, so a half-finished file can never be mistaken for a complete one.
- **Resumes** instead of starting over. It asks the server for the remaining bytes and appends them.
- **Retries** four times with increasing waits before giving up, and says what it is doing each time.

Tested against a simulated failure. The resume path was checked by cutting a transfer off part-way through: it picked up where it stopped and the final file was byte-for-byte correct, with no fragments left behind.

dataset.py — strip the units, multiply the examples

This is the most important file in the project, and the shortest. Everything the experiment claims rests on what happens here.

Job 1: take the units away

A methane picture holds numbers around 1850, in parts per billion. A carbon monoxide picture holds numbers around 0.030, in moles per square metre. You obviously cannot feed one to a detector trained on the other.

So for each picture separately, this file **subtracts the middle value and divides by the spread**. Both then become dimensionless maps centred on zero, saying only 'this pixel is high for this scene, that one is low'. It is the same move you make when you remove a baseline before comparing two instrument records.

Two details that matter:

- It uses the **median and the interquartile range**, not the mean and standard deviation. A bright plume filling part of the scene would drag the mean and inflate the standard deviation — the plume would partly erase itself. The median and IQR resist that.
- **Missing pixels become zero** after normalising, which is 'no anomaly'. This is a real modelling decision with a real consequence, described in the box below.

This choice caused a measurable problem. Filling gaps with zero leaves a flat, textureless block against noisy real data — a straight edge that never occurs in nature. In the finished results, tiles with more missing data score systematically lower, and that turned out to be one of the strongest effects in the whole analysis. It is the clearest example in this project of a small pipeline decision leaking into a scientific conclusion.

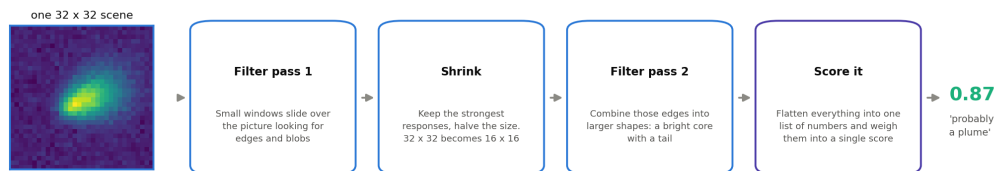
Job 2: make eight copies of every example

A plume can point whichever way the wind blows, and the satellite does not care which way is up. So every picture is rotated four ways and mirrored, turning 3,070 examples into 19,648. This is not a trick to inflate the numbers — it encodes a true fact about the physics, and it is what the original authors did too.

model.py — the detector

What happens inside the detector

A pattern-matcher that looks at shape, then a scorer that turns shape into a number.



The important part: the only thing entering on the left is a grid of numbers. There is no chemistry inside this box, no units, no knowledge of methane. It is a shape detector — which is exactly why it can be pointed at CO.

Forget the phrase 'neural network' — it does more harm than good here. What this does is closer to a **matched filter**: you slide a template across a signal and see where it fits. The difference is that nobody writes the templates. They are found automatically during training, by whatever happens to work.

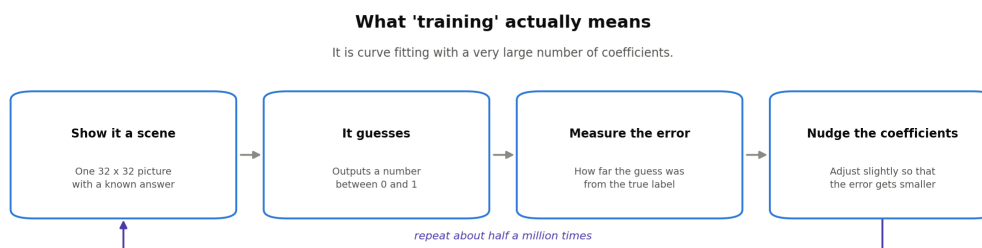
The four steps inside

- **Filter pass one.** Small windows slide across the 32 by 32 picture, each looking for one simple thing: a bright edge, a dark-to-light step, a compact blob.
- **Shrink.** Keep only the strongest responses and halve the size, 32 becomes 16. This throws away exact position and keeps the pattern.
- **Filter pass two.** Combine those simple features into larger ones — a bright core with a tail leading away, which is a plume seen from above.
- **Score.** Flatten everything into one long list of numbers and weigh them into a single number between 0 and 1.

About 590,000 numbers get adjusted during training. That sounds enormous, and it is the whole reason the thing works at all.

This is a reconstruction, not their model. Schuit et al. describe 'two convolutional blocks followed by two fully connected layers' on 32 by 32 single-channel scenes. Layer widths, window sizes, dropout and the optimiser are not stated in the paper, so those were chosen here. You are reproducing their approach, not their network, and that distinction has to be in anything you publish.

train.py — teach it what a plume looks like



Same idea as fitting a calibration curve to your analyser, with two differences: there are ~590,000 coefficients instead of two, and they are found by repeated small corrections rather than solved in one step.

Training is curve fitting with a very large number of coefficients. Show it a picture whose answer you know. It guesses a number. Measure how wrong the guess was. Nudge every coefficient slightly in whichever direction would have made the guess better. Repeat about half a million times. There is no insight in there — just a very large number of very small corrections.

The five stages it prints

Stage	What happens	Roughly how long
1 Read	Open the NetCDF files	seconds
2 Split	Set aside 20% as a test set, before making the rotated copies	instant
3 Normalise	Strip the units off ~20,000 pictures	a minute
4 Build	Create the network, count its coefficients	instant
5 Train	The loop above, one line of progress per batch	30-90 s per round

The one trap in the whole project. The split must happen BEFORE the rotations. Do it the other way round and a rotated copy of a training picture lands in the test set — the model has effectively already seen the answer, and your score becomes meaningless. The code does this correctly. Do not tidy it away.

What changed after you reported it looked frozen

Two stretches ran silently for minutes: the unit-stripping loop, and each training round which only printed at the end. Both now report continuously.

- **Progress per batch**, with running loss, speed and time remaining — and from the second round onwards, an estimate for the whole run. Within two minutes you know whether you are looking at twenty minutes or two hours.
- **Ctrl-C is safe.** The best model so far is saved after every improving round, so stopping early keeps your result and still prints the summary.
- **--quick** runs 600 pictures for three rounds in about three minutes, to prove the pipeline before you commit to the real thing.

- **Works when piped.** In a terminal it redraws one line; in a notebook or log file it prints a fresh line every 20% instead.

fetch_co_tiles.py — get the carbon monoxide pictures

This file caused the most trouble and went through the most versions. It is worth understanding what it does and what went wrong.

What it does now

- Picks **N days spread evenly** across your date range — the same days for every site.
- For each site and each day, asks Google Earth Engine for that day's carbon monoxide over a box about 220 km across, centred on the facility.
- Also fetches **nitrogen dioxide** and **MODIS fire radiative power** for the same place and day, which is what the fire filter needs.
- Rejects a tile if less than 60% of it survived the quality filter.
- Saves everything into one **.npz** file, plus a summary of what failed.

Four things went wrong, and what each taught

Problem	Cause	Fix and consequence
Every NO ₂ and fire lookup failed	The date was passed as 20190101. Earth Engine needs 2019-01-01. CO never touches the date, which is why only these two failed.	Correct date formatting. The fire filter became possible at all.
All 87 tiles from 3 days	The code took the first 40 orbits. At 14 orbits a day that is 72 hours.	Sample evenly across the range. Coverage went from 3 days to 18.
Region tied to season	Earth Engine gives every orbit a near-global footprint, so the same orbits were picked for every site, and only a few covered any given one.	Choose the days first, then ask for each day's data. Yield rose from 4% to 72% and every site now shares one calendar.
Silent failures	One bad date printed thousands of identical error lines.	Count failures, print one summary, and distinguish 'no data that day' from 'the call errored'.

The resolution number in the catalogue is not the resolution. Earth Engine lists 1113.2 metres for this product. That is the grid spacing — 0.01 degrees at the equator. The real information content is the native satellite footprint: 7.0 by 7.0 km at launch, 7.0 by 5.5 km since August 2019. The grid oversamples the truth about six times over. That is why the code samples at 7 km and not at 1.1 km.

fetch_co_l2.py — the same thing, done properly (optional)

You did not need this for the two-day result, and still do not. It is here because it makes an argument you can use.

A raw satellite file is an array of 4,172 rows along the flight track by 215 columns across the swath. That is the detector's own frame, and 'column' means literally which of the 215 detector columns took the measurement.

Why that frame matters

Borsdorff and colleagues showed in 2019 that TROPOMI carbon monoxide orbits carry a **striping artefact**: systematic differences that run along the flight direction, tied to detector column, exceeding 5 parts per billion, growing over the mission, cause still not fully understood — and **not corrected** in the operational product.

Now notice: a stripe is straight, elongated and coherent. So is a plume. Stripes are one of the artefact classes the second stage of the published method exists to reject. And they are only cleanly visible in the detector's own frame. Put the data on a latitude-longitude map and the stripes get smeared across a slanted orbit track and mixed into the background.

You saw this in your own data. The diagonal banding across your ArcelorMittal Gent and Sahara tiles is exactly this artefact, turned diagonal by the regridding. You can say you observed it rather than read about it.

What the file adds

- **You choose the quality threshold.** Earth Engine fixes it at 0.5 before you see the data. Here it is a setting.
- **A stripiness number for every tile** — the ratio of across-track to along-track pixel-to-pixel variation, following the metric in that paper. Around 1 means no preferred direction; well above 1 means striped. It cannot be computed from the gridded product at all.
- **Swath-edge warnings.** Pixels near the edge of the swath are much larger than the 7 km nadir value, so their scores deserve less trust.

The catch is access, not analysis: it does not download anything, and one orbit file is a few hundred megabytes from the Copernicus Data Space. Slicing native geometry is actually *easier* than regridding — no interpolation at all — but getting the files is the work.

predict_co.py — run the detector and judge it

Loads the trained detector, strips the units off every CO picture the same way, and produces a score from 0 to 1 for each. Then it does three more things.

It computes the combustion-efficiency ratio

Carbon monoxide comes from burning — *all* burning. A steel furnace makes it, so does a crop fire, a forest fire and every vehicle in a city. Van der Velde's 2021 result gives a way to tell them apart: hot, efficient, flaming combustion makes relatively more nitrogen dioxide; cool smouldering makes relatively more carbon monoxide. So the ratio of the two in the same scene tells you what kind of burning you are looking at.

One constant is deliberately left unset. The threshold separating smouldering from flaming depends on units, fuel, season and how long NO_2 survives in the air. There is no universal value. The code plots the ratio for fire scenes against non-fire scenes so you can read the threshold off your own data. Shipping a plausible-looking number you had not checked, to the man who published the method, would be worse than shipping none.

It draws two figures

- A **contact sheet** of the highest-scoring scenes. Look at this before believing any number — a score can separate two groups for entirely the wrong reason.
- A **calibration plot** of the $\text{CO}:\text{NO}_2$ ratio split by whether MODIS saw a fire, which is how you set that missing threshold honestly.

It writes co_detections.json

One record per tile: facility, date, score, whether it counted as a detection, the ratio, the fire radiative power, and a fire flag. This file is what all the later analysis reads.

browse_tiles.py — look at your data

Written last, and it immediately found something the summary statistics had hidden.

```
python src/browse_tiles.py --max-score 0.3 --out figures/rejected
python src/browse_tiles.py --site "Bhilai Steel Plant" --sort date
python src/browse_tiles.py --min-score 0.9 --out figures/confident
```

One image file per site. Each panel is labelled with the date, the score, how much of the tile was missing data, and a mark if MODIS saw a fire. It works with or without the scores file.

The part that matters most is the footer

Each sheet lists **the days on which that site produced no tile at all**. Those absences are data. The fetch drops a site-day silently when the tile is empty or mostly missing, and those drops are not random:

What	Lost	Why
Central Pacific	15 of 18 days	Dark ocean. The 2.3 micron retrieval needs surface reflectance it does not get.
Mumbai, Dolvi, Hazira	10-12 of 18	West-coast India — monsoon cloud.
Bokaro, Kosice	1 of 18	Inland and dry. Almost complete coverage.
June and August	50% and 54%	The monsoon, across all Indian sites.

This is a selection effect, and it points the same way as your main confound. What survives is the clear-sky subset, and clear sky is not evenly spread over your sites or your calendar. Your 'Indian steel' group is really inland eastern plants in the dry season — which is also where CO backgrounds are highest. A reviewer would find this in thirty seconds, so name it first.

Where the code came from

Schuit and colleagues released their **training data** but **not their source code** — the paper's assets page lists data DOIs and the WRF weather model, and no repository. So none of the Python here is theirs. Every line was written for this project.

What comes from the published work

- The 32 by 32 scene size, the two-block architecture description, training on the methane channel alone, the eightfold rotation and mirroring, the two-stage design, the 0.5 threshold, and the benchmark figures of 95.6% and 94.2%.
- The labelled scenes themselves, under a licence that requires attribution.
- From Leguijt et al. (2025): the European facility list and the 2019 study year.
- From van der Velde et al. (2021): the CO:NO₂ combustion-efficiency idea.

What is a judgement call

Layer widths and the optimiser. The normalisation scheme — the single most consequential choice in the project. Filling gaps with zero. Weighting the loss instead of resampling. Using the gridded product instead of raw. The 60% completeness threshold. **PROVENANCE.md lists these line by line.**

Before this goes on GitHub under your name.

Read every line, and cut what you cannot justify. Change what you disagree with — you know atmospheric data better than this scaffold does, and the normalisation and gap-filling are where your judgement should override it. Be ready to answer why the median and IQR rather than mean and standard deviation, why the split comes before the rotations, and why the fire threshold is unset.

And put a line in the README saying the scaffold was AI-assisted and what you did with it. That is ordinary practice now and costs nothing. Being asked later without having said so is the outcome worth avoiding.